implement memory-mapped file

0x10. requirement

> file-backed lazy virtual memory를 구현하라

0x20. DESIGN

0x21. outline; 큰 그림

1. 사용자 `mmap(len, prot, flags, fd, 0)` 호출
2. kernel이 인자를 검증하고, `TRAPFRAME` 아래 빈 주소 구간을 찾음
3. VMA 슬롯에 주소/길이/권한/flags/file reference를 저장하고 주소 반환
4. 사용자가 mapping 주소에 접근하면 page fault 발생
5. `trap.c`가 `mmapfault()` 호출
6. `mmapfault()`가 파일 내용을 읽어 물리 page를 채우고 PTE 등록
7. `munmap()` 또는 `exit()` 때 `MAP_SHARED | PROT_WRITE` page를 파일에 write-back하고 PTE/VMA/file reference 정리

mmap

- user program
  - -> mmap()
  - -> syscall
  - -> sys_mmap()
- 파일을 프로세스 주소 공간에 연결하는 VMA metadata 생성

mmapfault

- user program이 mmap 주소 접근
  - -> CPU page fault
  - -> trap
  - -> usertrap()
  - -> mmapfault()
- VMA metadata만 있던 mmap 영역을 실제 page table mapping으로 바꿈

munmap

- user program
  - -> munmap(addr, len)
  - -> syscall
  - -> sys_munmap()
- 프로세스 주소 공간에서 file-backed mapping을 제거
- memory 변경사항 -> file에 write-back

hook in fork

> mmap 계약을 child에게 복제하는 역할 수행

- copy the VMA array

hook in exit

> 프로세스가 아직 munmap하지 않은 mapping을 커널이 강제로 정리

- for each used VMA, run the unmap-with-writeback path.

0x22. `sys_mmap`

책임/의도

- VMA 등록만 함
- physical page allocate 안 함
- file read 안 함
- mapped virtual address를 user에게 return

1. validate args

- len – size to map or unmap, a positive PGSIZE multiple.
  1. 양수여야 함
  2. PGSIZE의 배수여야 함.
- prot – access permission: PROT_READ, PROT_WRITE, or both. Sets the PTE permission bits on the mapped pages.
  - 다음 안되는 경우를 제외하자.
  - 0
  - 알 수 없는 bit가 섞인 값
- flags – sharing mode, exactly one of MAP_SHARED (writes back to the file) or MAP_PRIVATE (changes stay process-local).
  - 이것도 다음만 가능
  - MAP_SHARED / MAP_PRIVATE
- fd – the open file to map. filedup bumps its refcount so the mapping survives a later close of the fd.
- offset – start position in the file to map.
  - 명세상 항상 0

2. find a free VMA slot

- 이 mapping을 저장할 metadata 공간을 찾음.

3. pick an unused VA below TRAPFRAME, above p->sz

- heap과 trapframe 사이의 빈 user virtual space를 고름.

4. fill in the VMA and return its addr.

- Increment ref count of file using filedup(f) when you fill the file filed in the VMA.
  - 왜? mapping이 fd의 생명에 묶이지 않게 함.
-

0x22. `mmapfault`

1. Find the VMA covering va. (Return -1 if not found).

- fault 난 주소가 mmap 영역인지 확인함.

2. Allocate a physical page and initialize it with zeros.

- file 내용을 담을 실제 memory page를 만듦.

3. Read the file data into the allocated page.

- (hint: Refer to the inode reading logic inside fileread())
- mapped file의 해당 offset 내용을 memory에 올림.

4. Translate prot to PTE permission bits, and map the physical page to the va.

- `PROT_READ/WRITE`를 page table permission으로 바꿈.

5. Return the physical address mapped to va.

0x23. `munmap`

1. Write back the data to the file if VMA is MAP_SHARED and free the physical page and clear the PTE

- If there is no PTE in the area (pages never faulted before) just skip the page.
- memory mapping을 끊으면서 변경사항과 memory를 정리함.

2. Adjust the VMA's addr, offset, length.

- 부분 unmap 후 남은 mapping의 범위를 고침.

3. Close the file and free the VMA slot if whole VMA gone

- mapping 전체가 사라졌을 때 metadata와 file ref를 정리함.

0x30. IMPL

0x31. sys_mmap

1. 먼저 요청 arg검증

```
// kernel/sysfile.c

// 1. validate args
if (len <= 0 || len % PGSIZE != 0)
    return -1;
if (prot == 0)
    return -1;
if (prot & ~(PROT_READ | PROT_WRITE))
    return -1;
if (flags != MAP_SHARED && flags != MAP_PRIVATE)
    return -1;
if ((prot & PROT_READ) && !f->readable)
    return -1;
if ((prot & PROT_WRITE) && flags == MAP_SHARED && !f->writable)
    return -1;
if (offset != 0)
    return -1;
```

2. 주소 선택은 heap 위, `TRAPFRAME` 아래 영역에서 아래쪽으로 탐색한다. 기존 VMA와 겹치면 피한다.

```
// kernel/sysfile.c
static int
vmaconflict(struct proc *p, uint64 addr, uint64 len)
{
    for(int i = 0; i < NVMA; i++){
        struct vma *v = &p->vmas[i];
        if(!v->used)
            continue;
        if(addr < v->addr + v->length && v->addr < addr + len)
            return 1;
    }
    return 0;
}
```

- 위와 같이 충돌 여부를 확인할 수 있는 함수를 추가

```
// 3. pick an unused VA below TRAPFRAME, above p->sz
uint64 va_mmap = 0;
uint64 heap_top = PGROUNDUP(p->sz);
for(uint64 a = TRAPFRAME - len; ; a -= PGSIZE){
    if(a < heap_top) break;
    if(!vmaconflict(p, a, len)){
        va_mmap = a;
        break;
    }
    if(a < PGSIZE) break;
}
if(va_mmap == 0) return -1;
```

3. 선택된 주소와 파일을 VMA에 저장한다.
    - `filedup()`을 씀 -> 사용자가 원래 fd를 닫아도 mapping이 파일 참조를 유지한다.

```
// 4. fill in the VMA and return its addr
vv->addr = va_mmap;
vv->f = filedup(f);
vv->flags = flags;
vv->length = len;
vv->offset = offset;
vv->prot = prot;
vv->used = 1;

return va_mmap;
```

0x32. mmapfault

- `mmap()` 직후에는 PTE가 없으므로, 사용자가 해당 주소를 읽거나 쓰면 page fault가 발생한다.
- `trap.c`는 load page fault(13)와 store page fault(15)를 잡아 lazy `sbrk` 경로를 먼저 시도하고, 실패하면 `mmapfault()`를 호출한다.

```
// kernel/trap.c
} else if(r_scause() == 13 || r_scause() == 15){
    uint64 stval = r_stval(); // fault가 난 주소임.
    if(vmfault(p->pagetable, stval, (r_scause() == 13) ? 1 : 0) != 0){
    // lazily-allocated heap page handled
} else if(mmapfault(stval) == 0){
    // mmap lazy fault handled
} else {
    printf("usertrap(): unexpected scause 0x%lx pid=%d\n", r_scause(), p->pid);
    printf(" sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
    setkilled(p);
}
```

- `mmapfault()`는 fault 주소를 page boundary로 내린 뒤, 그 주소를 포함하는 VMA를 찾는다.

```
int
mmapfault(uint64 va)
{
    struct proc *p = myproc();
    struct vma *v = 0;
    uint64 va0 = PGROUNDDOWN(va);

    // find the vma which cover the va
    for(struct vma *vv = p->vmas; vv < &p->vmas[NVMA]; vv++){
        if (vv->used && vv->addr <= va0 && va0 < vv->addr + vv->length) {
            v = vv;
            break;
        }
    }
    if (v == 0)
        return -1;
    if (ismapped(p->pagetable, va0))
        return -1;
```

- 해당 VMA가 있으면 물리 page를 할당하고 0으로 초기화한 뒤, 파일에서 해당 offset의 내용을 읽어온다. 먼저 0으로 채우기 때문에 파일 끝 이후의 나머지 page 영역은 자연스럽게 zero-fill된다.

```
    // Allocate a physical page and initialize it with zeros.
    void *pa = kalloc();
    if (pa == 0)
        return -1;
    memset(pa, 0, PGSIZE);

    // Read the file data into the allocated page.
    ilock(v->f->ip);
    int r = readi(v->f->ip, 0, (uint64)pa, v->offset + (va0 - v->addr), PGSIZE);
    iunlock(v->f->ip);

    if(r < 0){
        kfree(pa);
        return -1;
    }
```

- 마지막으로 VMA의 `prot` 값을 PTE 권한으로 변환해서 user page table에 매핑한다.

```
    // Translate prot to PTE permission bits, and map the physical page to the va.
    int perm = PTE_U;
    if(v->prot & PROT_READ)
        perm |= PTE_R;
    if(v->prot & PROT_WRITE)
        perm |= PTE_W;

    // Return the physical address mapped to va.
    if(mappages(p->pagetable, va0, PGSIZE, (uint64)pa, perm) < 0){
        kfree(pa);
        return -1;
```

```
    }
    return 0;
```

- 이 방식의 결과는 다음과 같다.
  - `PROT_READ` mapping은 `PTE_R`만 갖는다.
  - `PROT_WRITE` mapping은 `PTE_W`도 갖는다.
  - `PROT_EXEC`는 정의만 있고 `sys_mmap()`에서 거절된다.
- read-only mapping에 쓰면 처음에는 read-only PTE가 만들어지고, 같은 store 명령이 다시 fault를 내면서 프로세스가 kill된다.

0x33. munmap

- `sys_munmap()`은 인자를 받은 뒤 내부 함수 `do_munmap()`에 맡긴다.
  - 이 내부 함수는 exit 경로에서도 재사용되도록 일부로 분리했음!

```
// kernel/sysfile.c

uint64
sys_munmap(void)
{
    uint64 addr;
    int len;

    argaddr(0, &addr);
    argint(1, &len);

    return do_munmap(addr, len);
}
```

- `do_munmap()`은 page 정렬과 overflow를 검사하고, 요청 범위를 완전히 포함하는 VMA를 찾는다.

```
// kernel/sysfile.c
int
do_munmap(uint64 addr, int len)
{
    if(len <= 0 || addr % PGSIZE || len % PGSIZE)
        return -1;

    uint64 end = addr + len;
    if(end < addr)
        return -1;

    struct proc *p = myproc();
    struct vma *v = 0;

    // 1. Write back the data to the file if VMA is MAP_SHARED and free the physical page and clear the PTE.
    for (struct vma *vv = p->vmas; vv < &p->vmas[NVMA]; vv++) {
        if (!vv->used)
            continue;

        if(vv->addr <= addr && end <= vv->addr + vv->length) {
            v = vv;
            break;
        }
    }
    if(v == 0) return -1;
```

- 명세에서 VMA 중간을 뚫는 unmap을 금지하고, 앞부분 또는 뒷부분 unmap만 허용한다.

```
    uint64 vend = v->addr + v->length;
    if(addr != v->addr && end != vend) // 가운데 뚫는거 금지
        return -1;
```

- `MAP_SHARED | PROT_WRITE` mapping이면 실제로 fault가 나서 물리 page가 붙은 page만 파일에 write-back한다. `MAP_PRIVATE`는 파일에 반영하지 않는다

```
    if (v->flags & MAP_SHARED && v->prot & PROT_WRITE) {
        for(uint64 a = addr; a < end; a += PGSIZE){
            if(ismapped(p->pagetable, a)){ // 여기 write-back
                filewrite(v->f, a, PGSIZE);
```

```
            }
        }
    }
    uvmunmap(p->pagetable, addr, len / PGSIZE, 1);
```

- 이때 uvmunmap?
  - lazy mapping 특성상 아직 fault가 나지 않은 page는 PTE가 없다. 이를 안전하게 처리하기 위해 `uvmunmap()`은 없는 PTE와 invalid PTE를 panic 하지 않고 건너뛴다.

```
// Remove npages of mappings starting from va. va must be
// page-aligned. It's OK if the mappings don't exist.
// Optionally free the physical memory.
void
uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if((va % PGSIZE) != 0)
        panic("uvmunmap: not aligned");

    for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
        if((pte = walk(pagetable, a, 0)) == 0) // leaf page table entry allocated?
            continue;

        if((*pte & PTE_V) == 0) // has physical page been allocated?
            continue;

        if(do_free){
            uint64 pa = PTE2PA(*pte);
            kfree((void*)pa);
        }
        *pte = 0;
    }
}
```

- 해제 후에는 VMA 메타데이터를 조정한다. 전체 해제면 파일 참조를 닫고 슬롯을 비운다. 앞부분 해제면 시작 주소와 파일 offset을 앞으로 밀고, 뒷부분 해제면 길이만 줄인다.

```
// 2. Adjust the VMA's addr, offset, length.
if(addr == v->addr && len == v->length){
    fileclose(v->f); // Close the file and free the VMA slot if whole VMA gone
    v->used = 0;
    v->addr = 0;
    v->length = 0;
    v->offset = 0;
    v->prot = 0;
    v->flags = 0;
    v->f = 0;
} else if(addr == v->addr){
    // 앞부분 unmap
    v->addr += len;
    v->offset += len;
    v->length -= len;
    } else {
    // 뒷부분 unmap
    v->length -= len;
    }
    return 0;
```

0x34. fork/exit 관리

- `fork()`에서는 heap 영역만 `uvmcopy()`로 복사한다.
  - mmap 영역은 `p->sz` 위쪽에 잡히므로 실제 mapped page가 부모에게 있더라도 child page table로 복사되지 않는다.
  - 대신 VMA 메타데이터를 복사하고 파일 참조를 `filedup()`한다. 따라서 child는 같은 가상주소 범위를 갖지만, 실제 page는 child가 접근할 때 다시 파일에서 lazy load된다.

```
// kernel/proc.c
// Copy user memory from parent to child.
```

```
if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
    freeproc(np);
    release(&np->lock);
    return -1;
}
np->sz = p->sz;

// hook for memory-mapped files

for(int i = 0; i < NVMA; i++){
    if(p->vmas[i].used){
        np->vmas[i] = p->vmas[i];
        np->vmas[i].f = filedup(p->vmas[i].f);
    }
}
```

- 프로세스 종료 시에는 열린 파일 descriptor를 닫기 전에 모든 VMA를 `do_munmap()`으로 해제한다.
  - 이때 `MAP_SHARED | PROT_WRITE` 영역은 동일한 write-back 경로를 탄다

```
void
kexit(int status)

{
    struct proc *p = myproc();

    if(p == initproc) panic("init exiting");

    // hoOOok for memory-mapped files

    for(int i = 0; i < NVMA; i++){
        struct vma *v = &p->vmas[i];
        if(v->used)
            do_munmap(v->addr, v->length);
    }
// 참고------- 여기서 do_unmap은 아까의 그것이다.
```

0x40. PS

> 궁금증 해소 및 trouble shooting

0x41. VMA? 왜 필요함?

- page table은 다음만 함
  - virtual address → physical address permission
- 그런데 page fault가 나면 OS는 더 많은 걸 알아야 한다.
  - 이 address가 stack인가? heap인가?
  - mmap된 file인가?
  - 읽기 가능한가?쓰기 가능한가?
  - file의 어느 offset과 연결되는가?
- 이 정보를 들고 있는 게 `VMA`다.

0x42. 어떻게 disk에서 파일 정보를 가져오는가?

OS 구조

file
→ inode
→ file block
→ disk block mapping
→ buffer cache
→ disk driver
→ real disk

- process는 file을 byte array처럼 본다.
- OS는 file을 block list처럼 본다.
- disk는 block number만 안다.

file layer

> file = 연속된 byte array

inode layer

- OS는 file을 `inode`로 본다.
- inode = file metadata + disk block 위치를 찾는 entry point

block mapping layer

- file 안의 logical block number→ real disk block number

buffer cache

- disk block을 바로 목적지로 보내지 않는다.
- OS는 먼저 disk block을 `buffer cache`에 올린다: buffer cache의 bp->data→ user/kernel memory
  - 다음 목적을 위해서
  1. cache: 같은 block을 또 읽으면 disk I/O를 줄임
  2. lock: 같은 block을 여러 process가 동시에 만지지 못하게 함
  3. block unit: disk I/O의 기본 단위를 OS object로 감쌈
- disk block 하나의 memory copy + lock + metadata

disk driver

- 진짜 disk와 말하는 건 여기

```
virtio_disk_rw(b->dev, b->blockno)
```

- 의미: b->dev, b->blockno에 해당하는 disk block을 읽어서b->data에 채워라

`either_copyout`

```
either_copyout(user_dst, dst, bp->data + off % BSIZE, m)
```

- buffer cache에 올라온 disk block 중 필요한 byte만 목적지로 copy
- 즉 disk에서 읽은 전체 block 중, user가 원하는 부분만 잘라서 보낸다.

0x50. result

0x51. mmaptest

```
qemu-system-riscv64 -machine virt -bios none -kernel
k-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ./mmaptest
test basic mmap
test basic mmap: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test lazy access
test lazy access: OK
test mmap two files
test mmap two files: OK
test fork
test fork: OK
test munmap prevents access
usertrap(): unexpected scause 0xd pid=5
            sepc=0xa98 stval=0x3fffffd000
usertrap(): unexpected scause 0xd pid=6
            sepc=0xb12 stval=0x3fffffc000
test munmap prevents access: OK
test writes to read-only mapped memory
usertrap(): unexpected scause 0xf pid=7
            sepc=0xc68 stval=0x3fffffc000
test writes to read-only mapped memory: OK
mmaptest: all tests succeeded
$
```

0x52. usertests -q

```
            sepc=0x46f0 stval=0x3c012000
usertrap(): unexpected scause 0xf pid=6642
            sepc=0x46f0 stval=0x3d012000
usertrap(): unexpected scause 0xf pid=6643
            sepc=0x46f0 stval=0x3e012000
usertrap(): unexpected scause 0xf pid=6644
            sepc=0x46f0 stval=0x3f012000
PASS
test lazy_copy: PASS
test lazy_sbrk: PASS
PASS ALL TESTS
$
```